

See also

[Configuration \[button\]](#)

Methods and Read-Only Attributes of the pe

_Methods and Read-Only Attributes of the Place in the Store [PE]

The *PE* (**production element**), i.e., the place in the *Store*, provides the *methods* and *read-only attributes* listed in the table of contents to the left.

Cont [SimTalk] - pe, Store

Returns the *MU*, which is located at the top-most position on the designated storage place (production element) of the *Store* designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.pe(X:integer, Y:integer).Cont → object  
<Path>[X:integer, Y:integer].Cont → object
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the position of the storage place (on the production element) in the **X-Dimension**.
- The parameter *Y* of data type *integer* designates the position of the storage place (on the production element) in the **Y-Dimension**.

Return Value

The return value has the data type *object*.

Example

```
MyStore.pe(1,1).Cont.move(Station)  
MyStore[1,1].Cont.move(Station)
```

See also

[X-Dimension \[Store\]](#)

[Y-Dimension \[Store\]](#)

exitBlockList [SimTalk] - Store

Returns an array containing the *MUs*, which are going to exit the *Store* designated by <Path> and wait for a *Worker* to pick them up.

Remarks

Specify the optional parameter *ExitBlockingList* of data type *table* to write the objects contained in the **Exit Blocking List** into this list.

For the *Store* the **Exit Blocking List** also contains the position of the place for which the part is entered. If you need this information, you have to specify a table instead of writing the data to an *array*.

Do not specify the optional parameter *ExitBlockingList* to return an *array* containing the objects contained in the **Exit Blocking List**. The *array* is one-dimensional and only contains the objects in the blocking list. If you need the start times of the blockages, which are contained in the blocking table, you can query them from the object with the attribute *BlockingStarttime*.

Type

Method

Syntax

```
<Path>.exitBlockList([ExitBlockingList:table]) -> void/object[]
```

Parameter

The optional parameter *ExitBlockingList* of data type *table* designates the name of the table. This can be a *DataTable* or a local variable, into which the method writes the values.

Plant Simulation automatically generates the format of this table with two columns. Column 1 has the data type *object* and contains the *MUs* that unsuccessfully tried to exit the object. Column 2 contains the simulation time at which the *MUs* attempted to enter the object.

Return Value

The return value is an *array* of data type *object*.

Example

```
var musToExit := Store.exitBlockList // returns an array of MUs, which  
are going to exit  
var t:table
```

```
Store.exitBlockList(t)           // fills the table with MUs, which  
are going to exit
```

SimTalk

[BlockingStarttime \[SimTalk\]](#)

[exitBlockList \[SimTalk\] - material flow objects](#)

[exitBlockList \[SimTalk\] - lane A or B](#)

See also

[Exit Blocking List](#)

[Check the Contents List of the Stations](#)

getStackHeight [SimTalk] - Store

Returns the height of the stack in meters on the designated storage place in the *Store* designated by <Path>.

Remarks

getStackHeight also applies to places on *pallets/Containers* and to *Transporters* with a loading space of type **Store**.

Note

Here the height of the stack designates its physical height, not the number of parts on the stack.

If you output the value with the method *print*, the unit is the unit you selected under **Preferences/Model Settings > Unit > Length**.

Type

Read-only attribute

Syntax

```
<Path>.pe(X:integer, Y:integer).getStackHeight → length  
<Path>[X:integer, Y:integer].getStackHeight → length
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the position of the storage place (on the production element) in the **X-Dimension**.
- The parameter *Y* of data type *integer* designates the position of the storage place (on the production element) in the **Y-Dimension**.

Return Value

The return value has the data type *length*.

Example

```
print MyStore.pe(2,1).getStackHeight  
print MyStore[2,1].getStackHeight
```

SimTalk

[ZDim \[SimTalk\] - Store](#)

See also

[Stack Parts in the Store](#)

[Length \[preferences\]](#)

[X-Dimension \[Store\]](#)

[Y-Dimension \[Store\]](#)

mu [SimTalk] - PE, Store

Returns all stacked *MUs* on the designated storage place (on the production element) in the *Store* designated by <Path>.

Remarks

The method *mu* accesses the *MUs* on the stack if the **Z-Dimension** is greater than 1.

The index does not signify the order of entry.

You can query the greatest index with the read-only attribute *NumMU*.

Type

Method

Syntax

```
<Path>.pe(X:integer, Y:integer).mu(MU:integer) → object  
<Path>[X:integer, Y:integer].mu(MU:integer) → object
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the position of the storage place in the **X-Dimension**.
- The parameter *Y* of data type *integer* designates the position of the storage place in the **Y-Dimension**.
- The parameter *mu* of data type *integer* designates the position of the storage place in the **Z-Dimension**. Specify -1 to return the *MU* at the bottom of the stack. It returns VOID if the object is empty.

Default Value of the Parameter

The default value for the parameter *Number* is 1.

Note

The last *MU* is always the *MU* that is located on the bottom storage place, as a rule this is the *MU* that was moved onto the object first.

Return Value

The return value has the data type *object*.

Examples

```
print MyStore.pe(5,5).mu(1) // returns the topmost MU on the stack
                             // corresponds to print MyStore.pe(5,5).cont
print MyStore[5,5].mu(2)    // returns the second MU from the top on the
stack
print MyStore[5,5].mu(3)    // returns the MU at the bottom of a stack with
3 parts
```

```
print MyStore[1,1].MU(1)    // returns the topmost MU on the stack
print MyStore[1,1].MU(-1)   // returns the MU at the bottom of the stack
```

SimTalk

[NumMU \[SimTalk\] - PE, Store](#)

See also

[X-Dimension \[Store\]](#)

[Y-Dimension \[Store\]](#)

[Z-Dimension \[Store\]](#)

NumMU [SimTalk] - PE, Store

Returns the number of *MUs*, which are located on the *PE* (production element) in the *Store* designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.pe(X:integer, Y:integer).NumMU → integer  
<Path>[X:integer, Y:integer].NumMU → integer
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the position of the storage place in the **X-Dimension**.
- The parameter *Y* of data type *integer* designates the position of the storage place in the **Y-Dimension**.

Return Value

The return value has the data type *integer*.

Example

```
print MyStore.pe(1,1).NumMU  
print MyStore[1,1].NumMU
```

See also

[X-Dimension \[Store\]](#)

[Y-Dimension \[Store\]](#)

Read-Only Attributes of the Store

_Read-Only Attributes of the Store

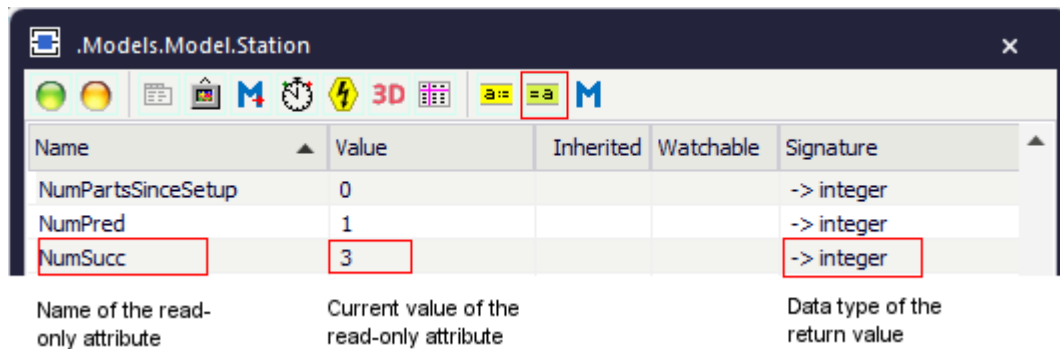
The *Store* provides:

- The *read-only attributes* listed in the table of contents to the left.
- The [_Read-Only Attributes of All Objects](#).

- The **Read-Only Attributes of the Material Flow Objects**.

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



Name	Value	Inherited	Watchable	Signature
NumPartsSinceSetup	0			-> integer
NumPred	1			-> integer
NumSucc	3			-> integer

Name of the read-only attribute Current value of the read-only attribute Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

To query the value of a *read-only attribute*, you might, for example, type:

```
print Store.Capacity
```